Solderbench: An open-source synthetic benchmark for PCB solder-joint defect inspection algorithm development

Qinmin Wang[a,1], Zixin Wang[b,2], Hongyuan Wang[a], Zongzhong Wang[c,*]

[a] *School of Computer Science and Artificial Intelligence, Guangdong University of Science and Technology, Dongguan, China*
[b] *School of Electronic and Communication Engineering, Tianjin Normal University, Tianjin, China*
[c] *School of Electromechanical Engineering, Guangdong University of Technology, Guangzhou, China*

Abstract

Automated optical inspection (AOI) of solder joints is a critical quality-control step in printed circuit board (PCB) assembly, and three-dimensional (3D) point-cloud inspection is increasingly adopted because solder quality is essentially a volumetric property. However, publicly available 3D solder-joint datasets are essentially nonexistent: industrial scan data are proprietary, so new inspection algorithms are difficult to compare fairly. We present Solderbench, an open-source, pure-Python synthetic benchmark for developing and evaluating 3D solder-joint defect inspection algorithms. Solderbench provides (i) a parametric point-cloud generator that simulates five joint classes (good, insufficient, excessive, bridging, and collapse) with more than fifteen controllable parameters and seed-level, point-wise reproducibility; (ii) a seven-dimensional geometric feature extractor; (iii) six baseline classifiers, including a re-implementation of the industrial volume-threshold rule S3DA; and (iv) automated comparison and ablation runners that export CSV, LaTeX, and figure artifacts. On a 1250-sample benchmark (seed 42), a random forest reaches 0.9440 accuracy and 0.9439 macro-F_1, and feature ablation identifies height variance as the most informative descriptor. Synthetic results serve as a controlled, reproducible proxy for algorithm development, yet final validation must be performed using real-world production scan data.

Keywords: solder joint inspection; automated optical inspection; 3D point cloud; synthetic benchmark; defect classification; reproducible evaluation; PCB; synthetic dataset

1. Required metadata

2. Motivation and significance

Surface-mount technology (SMT) dominates modern printed-circuit-board (PCB) assembly, and solder-joint quality largely determines product reliability. AOI is the standard quality-control step on SMT lines [1, 2, 3]; classical 2D AOI and deep-learning 2D detectors [4] on public datasets such as DeepPCB [5] are mature, yet inspection is increasingly moving to 3D because solder quality is fundamentally volumetric [6, 7]. General 3D deep learning (PointNet/PointNet++ [8, 9], Open3D [10]) has advanced, but 3D solder-joint research remains hampered by the absence of public data: existing public PCB datasets are almost entirely 2D [5], and the few 3D studies rely on private, non-public in-house point-cloud datasets [6, 7], so algorithms cannot be compared fairly or reproduced. Solderbench closes this gap synthetically, simulating solder-joint point clouds from a parametric geometric model whose parameters (nominal joint size, noise, defect strength, bridging distance, collapse depth) are physically interpretable and user-controllable; synthetic generation yields costless labeled data of arbitrary size, deliberate control of class difficulty and sensor noise, and seed-level reproducibility.

*Corresponding author at School of Electromechanical Engineering, Guangdong University of Technology, Guangzhou, China. E-mail: wangzz@gdut.edu.cn
[1] E-mail: wangqinmin@gdust.edu.cn
[2] E-mail: 396514839@qq.com

Table 1: Code metadata (mandatory).

Nr.	Code metadata description	Value
C1	Current code version	v0.1.1
C2	Permanent link to code/repository used for this code version	https://github.com/wqinmin975/pcb
C3	Permanent link to reproducible capsule	10.5281/zenodo.22959193
C4	Legal code license	MIT License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python ($\geq$ 3.8)
C7	Compilation requirements, operating environments and dependencies	NumPy, SciPy, pandas, scikit-learn, and Matplotlib; OS-independent (no compilation, no GPU required)
C8	Link to developer documentation/manual	https://github.com/wqinmin975/pcb#readme
C9	Support email for questions	wangzz@gdut.edu.cn

3. Software description

3.1. Software architecture

Solderbench is a pure-Python package (about 1,000 source lines) with five feed-forward modules behind one public API: `generator.py` (parametric simulator), `features.py` (seven descriptors), `baselines.py` (six classifiers), `evaluate.py` (`run_comparison`/`run_ablation` runners exporting CSV, Booktabs-style LaTeX tables, and figures), and `data.py` (`load_dataset()`, the real-camera integration point). Stages exchange NumPy arrays and pandas DataFrames, so any stage is replaceable; synthetic clouds swap for segmented real scans at `load_dataset()` without downstream changes. The package ships a 15-test suite (shape and class-balance checks, same-seed reproducibility, feature validity, a Welch t-test that insufficient-solder volumes are significantly smaller, and fit-and-predict smoke tests), a `pyproject.toml`, and a `codemeta.json`; dependencies are only NumPy, SciPy, pandas, scikit-learn, and Matplotlib [14, 15, 16, 17]. All results use the versions pinned in `requirements.txt` (scikit-learn 1.9.0, NumPy 2.5.2, pandas 3.0.6, SciPy 1.18.0, Matplotlib 3.11.1; Python 3.13); the benchmark is fully deterministic from seed 42, so `python examples/run_benchmark.py` regenerates every table and figure.

3.2. Software functionalities

Generation. SolderJointGenerator simulates five SMT defect classes [6, 7]: good, insufficient (shrunk ellipsoid, 0.55–0.85 of nominal), excessive (inflated ellipsoid, mean inflation 1.3, plus a bump), bridging (small center-to-center distance plus a strip), and collapse (random top dent). Over fifteen parameters control n_{points}, noise, and the shrinkage, inflation, bridging, and collapse ranges; class ranges deliberately overlap so no classifier is perfect, and difficulty is tuned by widening or narrowing them. **Table 2** lists all generator parameters with their defaults and physical meaning.

Features. extract_features returns seven descriptors (Table 3): `vol` (voxel volume), `max_h`, `h_var`, `area`, `vol_area_ratio`, `convex_vol_ratio` (irregularity), and `min_dist` (bridging exposure). **Baselines and runners.** get_baselines returns six classifiers. S3DA reproduces the industrial volume-threshold rule [6] as a shallow decision tree (max_depth $= 6$); although trained on all seven features, its learned splits are dominated by the voxel-volume descriptor, so it behaves as a data-driven volume-threshold proxy whose decision threshold is *learned from the training split* (not hand-crafted). The other five are logistic regression, an RBF-SVM [12], kNN, gradient boosting, and a random forest [11]; all are scikit-learn estimators [13] with fixed random states. Crucially, **all six methods share the identical stratified 7:3 split and the same seven-feature input**, so the comparison is fair; S3DA is intentionally the weakest learner, representing the rule-based industrial baseline. run_comparison(X, y) evaluates all six under a stratified 7:3 split with 5-fold CV, returning accuracy, macro precision, recall and F1, and CV accuracy; run_ablation(X, y) does leave-one-feature-out and single-feature analysis anchored on the random forest (Table 4, B6) to isolate the contribution of each feature descriptor. Both export CSV and LaTeX, run_comparison also renders the best model's confusion matrix, and `python examples/run_benchmark.py` reproduces the full benchmark.

Table 2: Configurable parameters of `SolderJointGenerator` (defaults used for every reported result; all spatial quantities in mm).

Parameter	Default	Physical meaning
`n_points`	1500	fixed point count per cloud (simulates scanner resolution)
`noise`	0.005	Gaussian sensor-noise std. σ (mm)
`base_radius`	0.12	nominal joint horizontal radius a (mm)
`base_height`	0.10	nominal joint height c (mm)
`good_scale_std`	0.09	relative size variation of good joints
`insufficient_scale_range`	(0.55, 0.85)	shrink factor of insufficient joints (overlaps good lower tail)
`excess_scale_mean`	1.30	mean inflation factor of excessive joints
`excess_scale_std`	0.12	inflation variation (overlaps good upper tail)
`excess_bump_range`	(150, 350)	# extra points forming the excessive bump
`normal_dist_mean`	0.65	mean centre-to-centre spacing, non-bridging (mm)
`normal_dist_std`	0.07	spacing variation (mm)
`bridge_dist_mean`	0.34	mean spacing for bridging joints (abnormally small, mm)
`bridge_dist_std`	0.06	bridging spacing variation (mm)
`bridge_points_range`	(80, 180)	# points forming the solder bridge
`collapse_depth_range`	(0.28, 0.50)	dent depth as fraction of joint height

Table 3: The seven geometric descriptors returned by `extract_features`, their definitions, physical meaning, and computation. All coordinates are in millimetres (mm); the voxel edge is $\delta = 0.01$ mm.

Feature	Definition	Physical meaning	Computation (units)
`vol`	$N_{\text{vox}}\delta^3$	total solder volume	unique occupied voxels at δ=0.01 mm; mm^3
`max_h`	$\max_i z_i$	joint height	mm
`h_var`	$\text{Var}(z)$	height spread (dent / bump)	mm^2
`area`	$\text{Surf}(\text{Conv}(\mathcal{P}))$	joint surface extent	convex-hull area, mm^2
`vol_area_ratio`	vol / area	compactness	dimensionless
`convex_vol_ratio`	vol / $\text{Vol}(\text{Conv}(\mathcal{P}))$	irregularity (voids / dents)	dimensionless (≤ 1)
`min_dist`	$\min_{j\neq i}\|c_i - c_j\|$	bridging exposure	centre-to-centre distance (mm); synthetic: spacing distribution; real: distance between segmented joint centroids

Validity guarantees. Every cloud is resampled to exactly n_{points} points, preventing sampling-density leakage, ensuring volumetric features reflect geometry rather than point-sampling density. All randomness derives from one independent `RandomState` created at construction (not NumPy's global seed), so a given seed yields point-wise identical clouds, features, and tables — asserted by the test suite. There is no compiled code, GPU, or network access at runtime. The package is tested on Python 3.13 with the exact dependency versions pinned in `requirements.txt` (scikit-learn 1.9.0, NumPy 2.5.2, pandas 3.0.6, SciPy 1.18.0, Matplotlib 3.11.1); the archived Zenodo release v0.1.1 reproduces every table and figure bit-for-bit. The real-data integration point `load_dataset` is documented and covered by a unit test that runs the full pipeline on arbitrary user-supplied `X, y`.

4. Illustrative examples

4.1. Quickstart

Install the package and its dependencies first with `pip install -e .` (requires Python $\geq$ 3.8); the API below corresponds to version v0.1.1 and may change in later releases. The following end-to-end example generates a dataset, extracts features, and runs both experiments in eight lines:

```
from solderbench import (SolderJointGenerator, extract_features,
get_baselines, run_comparison, run_ablation)
# generate 1250 joints (5 classes x 250) at seed 42
gen = SolderJointGenerator(n_points=1500, noise=0.005, seed=42)
clouds, labels = gen.generate(n_per_class=250)   # 5 classes x 250 joints
X, y = gen.to_features(clouds, labels)           # 7-feature matrix + labels
results = run_comparison(X, y)   # 6 methods x Acc/P/R/F1 + 5-fold CV
print(results[["no", "name", "Accuracy", "F1"]])
abl = run_ablation(X, y)         # leave-one-out + single-feature dF1
print(abl.head(8))
```

4.2. Benchmark comparison

`examples/run_benchmark.py` produces the benchmark snapshot reported in Table 4. To establish reliability we repeat the full comparison on **five independent random seeds** (42, 123, 456, 789, 1011); the table reports the mean $\pm$ std of each metric across seeds. The seed-42 run (1250 samples, 5 classes $\times$ 250 joints, 1500 points per cloud, stratified 7:3 split, 5-fold CV on the training set) yields the single-run snapshot cited elsewhere (RF accuracy 0.9440, macro-F_1 0.9439). Three observations follow. First, the industrial rule S3DA (B1) is a strong baseline (mean accu-

Table 4: Baseline comparison across five independent random seeds (mean $\pm$ std; 1250 samples per seed, stratified 7:3 split, 5-fold CV on the training set). Best mean test metric in bold.

No.	Method	Acc.	Prec.	Rec.	F_1	CV
B1	S3DA (vol. rule)	0.9131±0.0156	0.9164±0.0153	0.9131±0.0156	0.9136±0.0153	0.9184±0.0149
B2	Logistic Regression	0.9579±0.0114	0.9588±0.0105	0.9579±0.0114	0.9578±0.0114	0.9625±0.0157
B3	SVM (RBF)	0.9611±0.0119	0.9616±0.0119	0.9611±0.0119	0.9610±0.0119	0.9696±0.0115
B4	KNN	0.9253±0.0178	0.9272±0.0184	0.9253±0.0178	0.9249±0.0184	0.9239±0.0176
B5	GradientBoosting	0.9477±0.0078	0.9491±0.0079	0.9477±0.0078	0.9476±0.0079	0.9557±0.0140
B6	**RandomForest**	**0.9531±0.0115**	**0.9538±0.0116**	**0.9531±0.0115**	**0.9530±0.0115**	0.9650±0.0127

racy 0.9131 across seeds; 0.8960 at seed 42), confirming that solder volume alone carries most of the class signal — exactly the physics the generator encodes. Second, most learning-based methods improve on the rule, although kNN remains statistically tied with it (its mean accuracy 0.9253 is within one std of S3DA's 0.9131). The random forest attains the best mean test accuracy (0.9531) and macro-F_1 (0.9530). Third, the random forest edges gradient boosting by only 0.0053 accuracy on average across seeds, while the inter-seed standard deviation of that gap (0.0065) *exceeds* the mean gap; the two are therefore statistically tied, and we do not claim RF strictly outperforms GB. Finally, test-set and cross-validation rankings do not fully coincide: the RBF SVM has the highest mean CV accuracy (0.9696) yet a lower held-out test accuracy (0.9611). This gap is consistent with mild overfitting but may also reflect variance of the particular held-out splits, which is exactly why Solderbench reports both protocols and why users must trust the independent test set rather than cross-validation alone. Figure 1 shows the confusion matrix of the random forest on

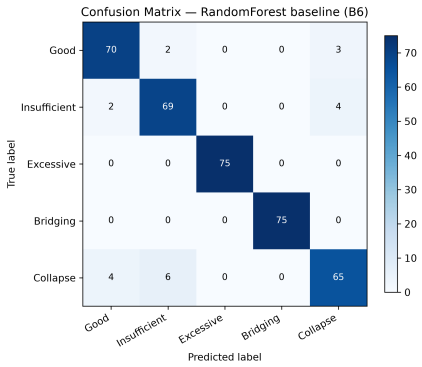


Figure 1: Confusion matrix for the Random Forest (B6) on the held-out test set (375 samples, seed = 42); the main confusion lies between collapse and the good and insufficient classes, while excessive solder incurs no test errors.

the held-out test set (375 samples, 21 misclassified). Bridging is perfectly separated (75/75), because the `min_dist` feature exposes the abnormally small inter-joint distance by construction. All 21 errors stay within the {good, insufficient, collapse} triad: collapse loses 4 samples to good and 6 to insufficient, and gains 3 good samples misclassified as collapse; good and insufficient exchange 2 samples each way, and insufficient loses a further 4 samples to collapse. The overall error therefore remains between collapse and the good and insufficient classes, because a shallow dent alters the height distribution only subtly — precisely why `h_var` — the most informative descriptor in the ablation (Table 7) — is needed to separate it; this mirrors the hard case reported for real 3D inspection systems [6, 7].

4.3. Representative samples, robustness, and physical validity

Figure 2 shows one representative synthetic point cloud per defect class (seed 42, 1500 points each). The generator is a parametric geometric model: every entry in Table 2 has a physical interpretation — the nominal joint radius a and height c set the joint scale in mm, `noise` is the sensor measurement standard deviation σ, and the shrinkage / inflation / bridging / collapse ranges express each defect as a controlled, physically meaningful deviation from the nominal geometry. This makes the benchmark's class difficulty directly tunable (widening the overlap ranges makes classes harder) and the defects interpretable, even though the clouds are not fitted to production scans.

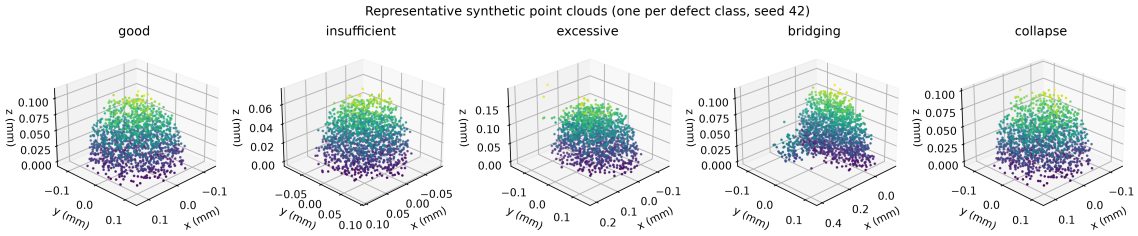


Figure 2: Representative synthetic point clouds, one per defect class (seed 42, 1500 points each; axes in mm). Good and insufficient differ mainly in scale; excessive adds a bump; bridging shows a low bridge to the neighbour; collapse shows a top dent.

To verify that the reported ranking is not an artefact of a single configuration, we repeat the full comparison under varying sensor noise and defect-strength settings (seed 42). Table 5 varies the noise std. σ over five levels $\sigma \in \{0.005, 0.01, 0.02, 0.05, 0.10\}$; accuracy falls monotonically and gracefully (RF 0.9440 $\rightarrow$ 0.7467) while the relative ordering of methods is preserved, confirming the benchmark is robust to sensor quality. Table 6 varies the class-overlap ranges: narrowing them (an easier task) raises RF accuracy to 0.9653, whereas widening them keeps RF at 0.9440 because the seven geometric features remain discriminative even under wide overlap — i.e., difficulty is controllable in the easy direction and the features are robust to aggressive overlap. Together these ablations show the conclusions hold across a range of physically meaningful generator settings.

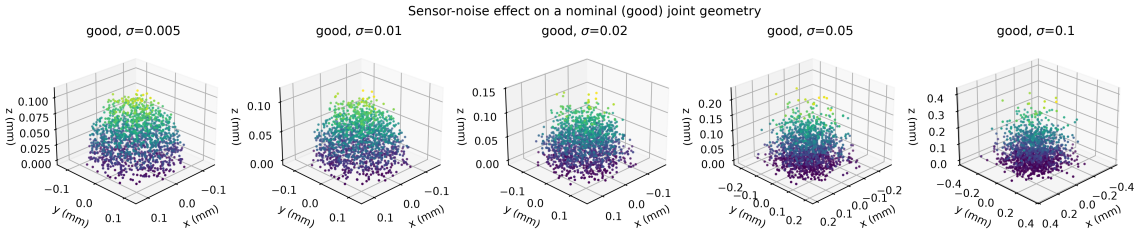


Figure 3: Effect of sensor noise on a nominal (good) joint: as σ grows the cloud becomes progressively rougher, degrading classifier accuracy (see Table 5).

Table 5: Robustness to sensor noise. Accuracy of the random forest (B6), the S3DA rule (B1) and gradient boosting (B5), and the span of all six methods, as a function of the Gaussian noise std. σ (seed 42).

σ	RF Acc.	S3DA Acc.	GB Acc.	Span_{acc}
0.005	0.9440	0.8960	0.9413	[0.8933, 0.9440]
0.010	0.9360	0.9013	0.9307	[0.8960, 0.9387]
0.020	0.9173	0.8667	0.9120	[0.8667, 0.9307]
0.050	0.8640	0.8293	0.8560	[0.8293, 0.8827]
0.100	0.7467	0.7200	0.7307	[0.6933, 0.7547]

Table 6: Robustness to defect-strength / class-overlap configuration. RF accuracy and the worst-method accuracy under a narrow (easier), default, and wide (harder) overlap of the class parameter ranges (seed 42).

Configuration	RF Acc.	S3DA Acc.	Min acc.
narrow (easier)	0.9653	0.9040	0.9040
default	0.9440	0.8960	0.8933
wide (harder)	0.9440	0.8960	0.8960

4.4. Feature ablation

Table 7 reports the leave-one-feature-out (LOO) ablation for the random forest (full-model macro-F_1 = 0.9439). The ablation has a clean physical reading. Height variance (`h_var`) is the single most valuable descriptor because

Table 7: Leave-one-feature-out ablation of the RandomForest model (macro-F_1; full model = 0.9439).

Removed feature	ΔF_1 **vs. full-model** F_1	**Interpretation**
h_var	-0.0459	Most informative: collapse dents and excessive bumps reshape the height distribution
vol_area_ratio	-0.0105	Compactness cue separating volumetric defects from nominal joints
min_dist	-0.0054	Small overall cost because bridging is nearly separable already
area	-0.0052	Correlated with vol; adds modest independent signal
convex_vol_ratio	-0.0025	Irregularity cue; partially redundant with h_var
vol	+0.0026	Slightly redundant once ratios are present
max_h	0.0000	Fully redundant given h_var and vol

the most error-prone defects — collapse (a dent) and insufficient solder (shrinkage) — primarily shift the height distribution, whereas excessive solder (a pronounced volume inflation) is in fact well separated, with zero test errors in Figure 1 because its volume signature is large and distinctive. Volume-only cues therefore saturate for the volumetric defects but miss the subtle height changes that distinguish collapse from good/insufficient. Single-feature runs corroborate this: the strongest single feature is `convex_vol_ratio` (macro-$F_1 = 0.6412$), which captures shape irregularity common to several defect types, while the weakest is `min_dist` (macro-$F_1 = 0.3616$), which is highly informative for bridging but carries no signal for the other four classes. The near-zero or positive deltas for `vol` and `max_h` indicate information redundancy rather than irrelevance: both are strongly correlated with the ratio features. Volume alone is in fact a strong single cue — it underpins the industrial S3DA rule (B1), which reaches 0.8960 accuracy using volume only. Removing `vol` slightly increases macro-F_1 within small numerical fluctuations; this is purely a statistical artifact given that volume information is already encoded within ratio-based descriptors.

Table 8: Selected representative single-feature macro-F_1 scores (RandomForest, seed = 42). Full results are programmatically exported by run_ablation.

Single feature	**macro-**F_1
`vol`	0.5494
`max_h`	0.5473
`h_var`	0.5808
`area`	0.6409
`vol_area_ratio`	0.5794
`convex_vol_ratio`	0.6412
`min_dist`	0.3616

5. Impact

Solderbench contributes to three communities. For inspection-algorithm research, it is the first public, reproducible 3D benchmark for solder-joint defect classification, replacing per-paper private datasets with a common task, protocol, and versioned reference numbers; new methods are ranked against six baselines with one command. For teaching, it is a self-contained geometric-ML case study that fits a short lab session and runs on any laptop, with each design decision doubling as a benchmark-methodology lesson. For reproducibility, seed-level determinism, a one-command script, machine-readable exports, a `codemeta.json` descriptor, and an MIT license make the software

trivially citable, archivable, and reusable. The path to production data is short: a user with a real 3D AOI camera replaces the body of `load_dataset` — or passes their own X, y — with code that reads scans, segments joints, and calls `extract_features`; all downstream experiments run unchanged. Beyond the classical-ML workflow, the generator's raw point clouds can be passed directly to external deep models (e.g., PointNet [8, 9]); Solderbench supports both scenarios.

6. Limitations

We state the limitations of Solderbench honestly. The generator is a geometric model, not a physics simulator: it does not model wetting dynamics, surface tension, oxidation, or realistic sensor artifacts (occlusion, multi-path reflections, structured-light failure); class distributions are plausible and overlapping, not fitted to production data. Three scope limits remain. First, the five classes are idealized single-fault geometries — compound defects (e.g., insufficient and collapsed together) are unsupported, although they occur on real-world production lines. Second, the clouds model only the individual joint, omitting substrate, pads, and pins, so board-level context is out of scope. Third, resampling every cloud to a fixed n_{points} simplifies real sensors, which return a variable point-count, and may alter the point-density distribution. **Consequently, Solderbench is suitable for algorithm prototyping, fair cross-method comparison, and descriptor-level analysis, but its metric values must not be read as predictions of real industrial AOI performance.** Synthetic validation cannot substitute for validation on real scans; the benchmark guarantees only that comparisons made on it are fair, controlled, and exactly reproducible — a much-needed property currently missing from this research field.

Data availability. The Solderbench source code and all reproduction scripts are released under the MIT license and archived on Zenodo (doi:`10.5281/zenodo.22959193`); the permanent GitHub repository is `https://github.com/wqinmin975/pcb`. The benchmark dataset is not stored as a fixed file but generated deterministically at runtime by running `python examples/run_benchmark.py`, which regenerates every table and figure in this paper.

7. Conclusions

We presented Solderbench, an open-source, pure-Python synthetic benchmark for 3D PCB solder-joint defect inspection with a parametric five-class generator, seed-level reproducibility, a seven-dimensional feature extractor, six baselines including the industrial S3DA rule, and automated comparison and ablation runners. On the reference benchmark (1250 samples, seed 42) a random forest reaches 0.9440 accuracy and 0.9439 macro-F_1. By removing the data-access barrier that fragments 3D solder-joint inspection, Solderbench makes development, comparison, and teaching open and reproducible. Future work: (i) fitting generator parameters to measured scans; (ii) adding sensor-artifact models; (iii) compound-defect taxonomy; (iv) real-data conversion scripts; (v) deep point-cloud model interfaces (e.g. PointNet, PointNet++); and (vi) preset difficulty templates.

8. Acknowledgements

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. Computations were performed with the Solderbench toolkit (MIT license).

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Qinmin Wang: Conceptualization, Methodology, Software, Writing – original draft. **Zixin Wang:** Software, Validation, Data curation. **Hongyuan Wang:** Resources, Supervision. **Zongzhong Wang:** Writing – review & editing, Supervision, Project administration (Corresponding author).

References

[1] Q. Ling, N.A.M. Isa, Printed circuit board defect detection methods based on image processing, machine learning and deep learning: A survey, IEEE Access 11 (2023) 15921-15944. https://doi.org/10.1109/ACCESS.2023.3245093

[2] Y. Zhou, M. Yuan, J. Zhang, G. Ding, S. Qin, Review of vision-based defect detection research and its perspectives for printed circuit board, Journal of Manufacturing Systems 70 (2023) 557-578. https://doi.org/10.1016/j.jmsy.2023.08.019

[3] M. Moganti, F. Ercal, Automatic PCB inspection systems, IEEE Potentials 14(3) (1995) 6-10. https://doi.org/10.1109/45.464686

[4] W. Dai, A. Mujeeb, M. Erdt, A. Sourin, Soldering defect detection in automatic optical inspection, Advanced Engineering Informatics 43 (2020) 101004. https://doi.org/10.1016/j.aei.2019.101004

[5] S. Tang, F. He, X. Huang, J. Yang, Online PCB defect detector on a new PCB defect dataset, arXiv preprint arXiv:1902.06197 (2019). https://doi.org/10.48550/arXiv.1902.06197

[6] Y. Xu, Z. Wang, J. Liang, Y. Zhang, J. Feng, W. Miao, J. Wang, G. Pu, S3DA: A 3D point cloud based PCB solder defect detection algorithm, in: S. Liu (Ed.), Software Fault Prevention, Verification, and Validation (SFPVV 2024), Lecture Notes in Computer Science, vol. 15393, Springer, 2025, pp. 115-130. https://doi.org/10.1007/978-981-96-1621-3_8

[7] Q. Hu, K. Hao, B. Wei, H. Li, An efficient solder joint defects method for 3D point clouds with double-flow region attention network, Advanced Engineering Informatics 52 (2022) 101608. https://doi.org/10.1016/j.aei.2022.101608

[8] C.R. Qi, H. Su, K. Mo, L.J. Guibas, PointNet: Deep learning on point sets for 3D classification and segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 652-660. https://doi.org/10.1109/CVPR.2017.16

[9] C.R. Qi, L. Yi, H. Su, L.J. Guibas, PointNet++: Deep hierarchical feature learning on point sets in a metric space, in: Advances in Neural Information Processing Systems 30 (NeurIPS), 2017, pp. 5099-5108. https://proceedings.neurips.cc/paper/2017/hash/d8bf84be3800d12f74d8b05e9b89836f-Abstract.html

[10] Q.-Y. Zhou, J. Park, V. Koltun, Open3D: A modern library for 3D data processing, arXiv preprint arXiv:1801.09847 (2018). https://arxiv.org/abs/1801.09847

[11] L. Breiman, Random forests, Machine Learning 45(1) (2001) 5-32. https://doi.org/10.1023/A:1010933404324

[12] C. Cortes, V. Vapnik, Support-vector networks, Machine Learning 20(3) (1995) 273-297. https://doi.org/10.1007/BF00994018

[13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: Machine learning in Python, Journal of Machine Learning Research 12 (2011) 2825-2830.

[14] C.R. Harris, K.J. Millman, S.J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N.J. Smith, et al., Array programming with NumPy, Nature 585 (2020) 357-362. https://doi.org/10.1038/s41586-020-2649-2

[15] P. Virtanen, R. Gommers, T.E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, et al., SciPy 1.0: Fundamental algorithms for scientific computing in Python, Nature Methods 17 (2020) 261-272. https://doi.org/10.1038/s41592-019-0686-2

[16] W. McKinney, Data structures for statistical computing in Python, in: Proceedings of the 9th Python in Science Conference, 2010, pp. 51-56. https://doi.org/10.25080/Majora-92bf1922-00a

[17] J.D. Hunter, Matplotlib: A 2D graphics environment, Computing in Science & Engineering 9(3) (2007) 90-95. https://doi.org/10.1109/MCSE.2007.55